
Binary Reverse Engineering Lab Material

Course: *Introduction to Vulnerability Research Lab*

Author name: *Dr. Bernard Lampe*

Due date: *October 31st, 2019*

Setup

1. The lab target is `binary\_re/targets/checks.c`, built without source access for the exercise (compile it yourself once, then work only from the binary):

```
1 clang -g checks.c -o checks           # instructor build; hide this source
```

2. It accepts a blob file: [magic 4 bytes] [1-byte version/flags] [1-byte key length] [key bytes] and prints `ACCESS GRANTED` only on full validation.
3. Reference blob behaviors (all verified on the reference build):

Blob	Content	Result
<code>valid_blob.bin</code>	magic, ver 2, 6-key blob	ACCESS GRANTED, exit 0
<code>antidebug_blob.bin</code>	flags nibble = 1	"debugger detected, exiting", exit 1
<code>badmagic_blob.bin</code>	magic = "XXX!"	rejected, exit 1

4. Tools: any disassembler (objdump at minimum; IDA/Binja/Ghidra/r2 preferred), a debugger (lldb on this host), Python for the transform work.

Exercise 1 — Static Triage

1. Run `file checks` and `otool -L checks`. What libc-level imports hint at the program's behavior before you read a single instruction?
2. `strings checks`: which strings survive, and what do the printed messages tell you about the control flow of `main`?
3. Disassemble only `check` (via `nm` tracing). Identify: how the version nibble is checked, how the flags nibble is used, and where `memcmp` is called.

Exercise 2 — Dynamic Discovery of the Tripwire

1. Run `./checks antidebug\_blob.bin` and observe the "debugger detected" message without any debugger attached: the tripwire is value-triggered, not debugger-triggered. Verify in the disassembly which nibble drives it.
2. Then trigger it under `lldb`: break on `printf` and inspect the flag byte on the stack. Confirm that clearing bit 0 of byte [4] bypasses it. Patch `antidebug\_blob.bin` into a clean blob and verify accepted-entry-set now passes this gate.

Exercise 3 — Decode the Key Transform

1. In the disassembly of the key loop, identify the three primitive operations applied per key byte (subtraction, rotation, XOR — which operand is the round-derived constant?).
2. Under lldb, break at the loop and record the key byte, the round number, and the decoded value for the first 3 rounds. Confirm the inversion:
 - (a) Decoded bytes 0–2 must equal `VR!`.
 - (b) The decoded key must sum to `0x1E1` (any key length the header allows).
3. In Python, write the inverse transform and produce a key blob for `klen = 6` (one exists: `valid\_blob.bin`). Confirm `./checks valid\_blob.bin` prints `ACCESS GRANTED`.
4. Produce a second blob with a different valid key length ($klen \geq 7$) to prove the sum invariant is length-generic, not patched.

Exercise 4 — Written Audit Report

1. Write the check function's spec as you recovered it: gate list, byte offsets, flags semantics, the per-byte transform, and the acceptance invariants.
2. Explain the anti-reversing options the author had available (per the lecture's Anti-Reversing section): which class does the flags-nibble tripwire belong to, and how robust was it?
3. Price the exercise in reversing-session terms (**Approach** section): what was the triage, the surface map, the automation done, and the dynamic confirmation? Which step gave you the most information per minute?

Deliverables

1. Annotated disassembly of `check` (gate list labeled by hand).
2. Your bypassed/patched `antidebug\_blob.bin` and confirmation run.
3. Python inversion script and two accepted key blobs.
4. The Exercise 4 written spec.